

Redis

Seven Databases in Seven Weeks — Capítulo 8

Base de datos en memoria con estructuras de datos avanzadas: strings, lists, sets, sorted sets y hashes.

Redis (REmote DIctionary Server) es una **base de datos en memoria** de código abierto que se destaca por **ofrecer estructuras de datos avanzadas** nativas, no solo cadenas de texto.

Arquitectura Principal

- **En memoria (In-memory):** Todos los datos residen en la memoria **RAM del servidor**, lo que permite **latencias de submilisegundos** y un rendimiento extremadamente alto para operaciones de lectura y escritura.
- **Un solo hilo (Single-threaded):** Redis **ejecuta los comandos** de los clientes **en un único hilo principal a través de un bucle de eventos** (event loop). Esto **elimina la necesidad de bloqueos complejos (locks)** y **garantiza la atomicidad**, evitando condiciones de carrera a nivel de instrucción.

Manejo de la Consistencia y Persistencia, se puede configurar a

- Redis prioriza el rendimiento y la consistencia de un solo nodo (CP en el teorema CAP para configuraciones de nodo único).
- **Snapshots (RDB):** Permite tomar **fotografías puntuales de la base de datos y guardarlas en disco**. Prioriza la velocidad de reinicio pero puede perder datos recientes en caso de caída.
- **Append-Only File (AOF):** **Registra cada operación de escritura** secuencialmente **en un archivo log**. Ofrece mayor durabilidad a costa de un ligero impacto en el rendimiento.
- **Transacciones Optimistas:** **usa comandos MULTI/EXEC para ejecutar bloques atómicos**. Si ocurre un **error de sintaxis**, la **transacción se aborta**; si ocurre un error lógico en tiempo de ejecución, Redis no realiza *rollback* (continúa con los demás comandos) para mantener la velocidad.

0. Setup — Redis con Docker

PowerShell (desde la carpeta Redis/)

```
docker-compose up -d

# Verificar que el container está corriendo:
docker ps

# Abrir la consola interactiva redis-cli:
docker exec -it redis-standalone redis-cli

# Cargar los datos de ejemplo (data.redis):
docker exec -i redis-standalone redis-cli < data.redis
```

NOTA: Todos los comandos del Día 1 al 3 se ejecutan dentro de *redis-cli* (prompt *127.0.0.1:6379>*). También se pueden correr directamente desde PowerShell con: *docker exec redis-standalone redis-cli COMANDO*

DÍA 1 — Strings, CRUD y Expiración

Redis **almacena todo como pares clave-valor**. El tipo más simple es el string, que puede contener texto, números o datos binarios. El libro empieza con operaciones básicas sobre strings.

1.1 Operaciones básicas sobre strings

MSET clave1 valor1 clave2 valor2: Permite **establecer múltiples pares clave-valor** en un solo comando atómico.

MGET clave1 clave2: **Recupera múltiples valores** simultáneamente.

Contexto: **MSET y MGET** son **fundamentales** para **reducir la latencia de red** (network round-trips) cuando se operan grandes volúmenes de datos.

redis-cli

```
# Hello, Redis!
SET hello world
GET hello

# Sobreescribir un valor:
SET hello "Hello, Redis!"
GET hello

# Verificar existencia y eliminar:
EXISTS hello
DEL hello
EXISTS hello

# SET y GET múltiples en una sola llamada:
MSET first "Tom" last "Burns" born 1949
MGET first last born
```

1.2 Contadores atómicos

INCR clave / DECR clave: **Incrementa o decrementa** un **valor** numérico entero en **1**. Debido a la **arquitectura** de un **solo hilo** de Redis, esta **operación** está **libre de condiciones de carrera** (race conditions).

redis-cli

```
SET count:visits 0
INCR count:visits
INCR count:visits
INCR count:visits
GET count:visits

# Incrementar por un valor fijo:
INCRBY count:visits 10
GET count:visits

# Decrementar:
DECR count:visits
```

```
DECRBY count:visits 5
```

1.3 Expiración de claves (TTL)

Redis **permite definir tiempo de vida (TTL) en cualquier clave** — ideal para cachés y sesiones temporales.

redis-cli

```
# Crear clave con expiración en segundos:
SET session:user1 "token-abc123"
EXPIRE session:user1 30

# Ver cuántos segundos quedan:
TTL session:user1

# SET con expiración directa (EX = segundos):
SET session:user2 "token-xyz789" EX 60
TTL session:user2

# Quitar la expiración (hacer persistente):
PERSIST session:user1
TTL session:user1

# -1 = sin expiración, -2 = clave no existe
```

DÍA 2 — Estructuras de Datos Avanzadas

Redis no es solo key-value simple: **soporta listas, sets, sorted sets y hashes de forma nativa**. Cada estructura tiene su propio conjunto de comandos optimizados.

2.1 Lists — Cola y Pila

Las listas de Redis son **linked lists**. Se pueden usar **como cola (FIFO) o pila (LIFO)**. El libro las usa para modelar colas de tareas.

RPUSH clave valor / LPUSH clave valor: Inserta un **elemento** en el extremo **derecho** (final) o **izquierdo** (inicio) de la lista.

RPOP clave / LPOP clave: Extrae y devuelve el **último** o el **primer elemento** de la lista, eliminándolo de la misma.

LRANGE clave inicio fin: Devuelve un **rango de elementos**. Sintaxis: Usar 0 -1 "desde el primer elemento hasta el último".

LLEN clave: Retorna la **longitud** actual (cantidad de elementos) de la lista de forma instantánea.

LINDEX clave indice: Recupera un **elemento** en una **posición específica** sin extraerlo de la lista.

redis-cli

```
# RPUSH: agregar al final (Right PUSH):
RPUSH todo "Buy groceries"
RPUSH todo "Clean the house"
RPUSH todo "Walk the dog"
RPUSH todo "Read Seven Databases"

# Ver la lista completa (índice 0 a -1 = todos):
LRANGE todo 0 -1

# Ver longitud:
LLEN todo

# LPUSH: agregar al principio:
LPUSH todo "Urgent task!"
LRANGE todo 0 -1

# Pop de cada extremo:
RPOP todo
LPOP todo
LRANGE todo 0 -1

# Acceder por índice:
LINDEX todo 0
LINDEX todo -1
```

2.2 Sets — Conjuntos sin duplicados

Los sets **almacenan valores únicos sin orden**. **Soportan operaciones de conjuntos** (intersección, unión, diferencia) — útiles para etiquetas, categorías, relaciones.

SADD clave valor: Añade uno o más **elementos** al conjunto. Si el valor ya existe, se ignora pasivamente.

SMEMBERS clave: Devuelve todos los miembros

SCARD clave: Devuelve la cardinalidad.

SISMEMBER clave valor: Verifica rápidamente si un **elemento específico pertenece** al conjunto (devuelve 1 si es verdadero, 0 si no).

SINTER clave1 clave2: Realiza una **intersección** matemática

SUNION clave1 clave2: Combina los conjuntos y devuelve una lista única sin duplicados.

SDIFF clave1 clave2: Resta el **segundo conjunto** al **primero**, devolviendo los elementos que están en el primero pero no en el segundo.

redis-cli

```
# Agregar miembros:
SADD tags:article:1 databases nosql redis
SADD tags:article:2 databases sql postgres
SADD tags:article:3 nosql mongodb redis

# Ver miembros:
SMEMBERS tags:article:1

# Cardinalidad (cantidad de elementos):
SCARD tags:article:1

# Verificar si un elemento está en el set:
SISMEMBER tags:article:1 redis
SISMEMBER tags:article:1 sql

# Intersección: artículos que tienen AMBAS etiquetas:
SINTER tags:article:1 tags:article:3

# Unión: todas las etiquetas de ambos artículos:
SUNION tags:article:1 tags:article:2

# Diferencia: etiquetas en art:1 pero NO en art:2:
SDIFF tags:article:1 tags:article:2
```

2.3 Sorted Sets — Rankings y Leaderboards

Es una **estructura híbrida** entre un **Set** y un **Hash**. Cada **elemento** único está **asociado a** un número de punto flotante llamado **score (puntaje)**. Redis **mantiene los elementos ordenados** automáticamente según su score **desde el momento de la inserción**. El libro los usa para leaderboards de videojuegos.

ZADD clave score miembro: Añade un **miembro** al **conjunto** con su respectivo puntaje.

ZRANGE clave inicio fin: Devuelve los **elementos** en un **rango** de posiciones **ordenados** de menor a mayor score.

ZREVRANGE clave inicio fin WITHSCORES: Igual que el anterior, pero en **orden descendente** (mayor a menor). El **modificador opcional WITHSCORES incluye el puntaje en la respuesta**.

ZSCORE clave miembro: Devuelve el **puntaje exacto asociado a un miembro** específico en tiempo constante.

ZRANK clave miembro / ZREVRANK clave miembro: Devuelve la **posición** (el índice basado en 0) del **miembro en el ranking**.

ZRANGEBYSCORE clave minimo maximo: Filtra y devuelve todos los miembros cuyo puntaje se encuentra dentro del rango numérico especificado.

ZINCRBY clave incremento miembro: Aumenta (o disminuye, si el valor es negativo) el puntaje de un miembro existente de manera atómica.

redis-cli

```
# Agregar miembros con score:
ZADD leaderboard 950 h4x0rjimduggan
ZADD leaderboard 850 pitythefool
ZADD leaderboard 720 alice
ZADD leaderboard 1100 redismaster
ZADD leaderboard 430 newbie

# Rango ascendente (score menor primero):
ZRANGE leaderboard 0 -1

# Rango descendente CON scores (top 3):
ZREVRANGE leaderboard 0 2 WITHSCORES

# Score de un miembro:
ZSCORE leaderboard h4x0rjimduggan

# Posición (rank) de un miembro (0-indexed):
ZRANK leaderboard h4x0rjimduggan
ZREVRANK leaderboard h4x0rjimduggan

# Miembros con score entre 800 y 1000:
ZRANGEBYSCORE leaderboard 800 1000 WITHSCORES

# Incrementar score (por ejemplo, nueva partida):
ZINCRBY leaderboard 200 alice
ZREVRANGE leaderboard 0 -1 WITHSCORES
```

2.4 Hashes — Objetos estructurados

Los hashes almacenan pares campo-valor dentro de una clave. Son eficientes para modelar objetos (usuarios, sesiones, configuraciones).

ZADD clave score miembro: Añade un miembro al conjunto con su respectivo puntaje.

ZRANGE clave inicio fin: Devuelve los elementos en un rango de posiciones ordenados de menor a mayor score.

ZREVRANGE clave inicio fin WITHSCORES: Igual que el anterior, pero en orden descendente (mayor a menor). El modificador opcional WITHSCORES incluye el puntaje en la respuesta.

ZSCORE clave miembro: Devuelve el puntaje exacto asociado a un miembro específico en tiempo constante.

ZRANK clave miembro / ZREVRANK clave miembro: Devuelve la posición (el índice basado en 0) del miembro en el ranking.

ZRANGEBYSCORE clave minimo maximo: Filtra y devuelve todos los miembros cuyo puntaje se encuentra dentro del rango numérico especificado.

ZINCRBY clave incremento miembro: Aumenta (o disminuye, si el valor es negativo) el puntaje de un miembro existente de manera atómica.

redis-cli

```
# Crear un perfil de usuario:
HSET user:1:profile name Alice age 30 city "Buenos Aires" role admin

# Obtener un campo:
HGET user:1:profile name

# Obtener todos los campos:
HGETALL user:1:profile

# Obtener solo keys o solo values:
HKEYS user:1:profile
HVALS user:1:profile

# Verificar si un campo existe:
HEXISTS user:1:profile email

# Agregar un campo:
HSET user:1:profile email alice@example.com
HGETALL user:1:profile

# Eliminar un campo:
HDEL user:1:profile role
HGETALL user:1:profile

# Incrementar un campo numérico:
HSET user:1:profile login_count 0
HINCRBY user:1:profile login_count 1
HGET user:1:profile login_count
```

DÍA 3 — Pub/Sub, Transacciones y Persistencia

3.1 Pub/Sub — Mensajería en tiempo real

Redis **implementa un sistema de mensajería publish/subscribe**. El libro lo usa para demostrar comunicación en tiempo real entre procesos.

SUBSCRIBE canal: Conecta el cliente a un **canal específico** y bloquea la terminal para escuchar eventos en tiempo real.

PUBLISH canal mensaje: Emite un **mensaje** hacia un **canal**. Devuelve el número de clientes que recibieron el mensaje.

PSUBSCRIBE patron: Permite **suscribirse** dinámicamente a **múltiples canales** utilizando patrones de tipo glob (ej. news* captura news.sports y news.tech).

NOTA: Pub/Sub requiere DOS terminales abiertas simultáneamente — una para el subscriber y otra para el publisher.

Terminal 1 — Subscriber

```
docker exec -it redis-standalone redis-cli

# Suscribirse a un canal:
SUBSCRIBE news

# (Queda esperando mensajes...)
# Ctrl+C para salir
```

Terminal 2 — Publisher

```
docker exec -it redis-standalone redis-cli

# Publicar un mensaje:
PUBLISH news "Breaking: Redis is awesome!"
PUBLISH news "Seven Databases chapter complete"

# El subscriber en Terminal 1 recibe los mensajes en tiempo real
```

Suscripción con patrón (glob)

```
# Suscribirse a todos los canales que empiecen con 'news':
PSUBSCRIBE news*

# Desde otro terminal, publicar en distintos canales:
PUBLISH news.sports "Gol de Argentina!"
PUBLISH news.tech   "Redis 8.0 released"
PUBLISH other       "Este NO llega (no coincide con news*)"
```

3.2 Transacciones — MULTI / EXEC

Redis ejecuta **transacciones como un bloque atómico**: todos los **comandos se encolan con MULTI** y se **ejecutan juntos con EXEC**. Nadie puede interrumpir entre medio. **DISCARD**: **Cancela la transacción** actual vaciando la cola de comandos antes de ejecutar EXEC.

redis-cli

```
# Iniciar transacción:
MULTI

# Comandos se encolan (responden QUEUED):
SET account:alice 100
SET account:bob 50
DECRBY account:alice 30
INCRBY account:bob 30

# Ejecutar todos en bloque atómico:
EXEC

# Verificar resultado:
GET account:alice
GET account:bob

# Cancelar una transacción antes de EXEC:
MULTI
SET temp "value"
DISCARD
EXISTS temp
```

3.3 Persistencia

Redis es in-memory pero soporta dos mecanismos de persistencia: RDB (snapshot) y AOF (append-only file). El docker-compose está configurado con AOF habilitado (--appendonly yes).

CONFIG GET parametro: muestra configuracion actual de persistencia

BGSAVE: fuerza a crear la snapshot de la base de datos

INFO seccion: Retorna métricas críticas de rendimiento, consumo de memoria, clientes conectados y replicación.

TYPE clave: **Inspecciona una clave y devuelve la estructura** de datos interna **que contiene** (string, list, set, zset o hash).

DBSIZE: Devuelve el **conteo total** rápido ($O(1)$) de **claves existentes** en la base de datos seleccionada.

FLUSHALL: **Borra absolutamente todas las claves** de todas las bases de datos de la instancia.

redis-cli

```
# Ver configuración actual de persistencia:
CONFIG GET save
CONFIG GET appendonly

# Forzar snapshot RDB (guardar en disco ahora):
BGSAVE
```

```

# Ver info de persistencia:
INFO persistence

# Estadísticas generales del servidor:
INFO server
INFO stats

# Ver todas las claves existentes:
KEYS *

# Buscar claves con patrón:
KEYS user:*
KEYS tags:*

# Tipo de dato de una clave:
TYPE todo
TYPE leaderboard
TYPE user:1:profile

# Número total de claves:
DBSIZE

# Vaciar toda la base de datos (CUIDADO):
# FLUSHALL

```

3.4 Scripting con Lua (EVAL)

Redis permite ejecutar scripts Lua atómicamente — sin que otros clientes puedan intercalarse durante la ejecución. El libro lo usa para operaciones complejas garantizadas.

redis-cli

```

# Script simple: retornar un valor:
EVAL "return 'Hello from Lua!'" 0

# Script con KEYS y ARGV:
EVAL "return redis.call('SET', KEYS[1], ARGV[1])" 1 mykey myvalue
GET mykey

# Script atómico: GET + SET en una sola operación:
EVAL "local v = redis.call('GET', KEYS[1]); redis.call('SET', KEYS[1], ARGV[1]); return v" 1 hello "new value"
GET hello

# Incremento condicional (solo si el valor actual es mayor a 0):
EVAL "if tonumber(redis.call('GET', KEYS[1])) > 0 then return redis.call('INCR', KEYS[1]) else return 0 end" 1 count:visits

```

Resumen de Comandos por Tipo de Dato

Strings	Comandos principales
SET / GET / DEL	Crear, leer, eliminar una clave
MSET / MGET	Operar sobre múltiples claves a la vez
INCR / INCRBY	Contador atómico (incremento)
DECR / DECRBY	Contador atómico (decremento)
EXPIRE / TTL / PERSIST	Definir o quitar tiempo de vida
APPEND / STRLEN	Concatenar o medir longitud del valor

Lists	Comandos principales
R PUSH / L PUSH	Agregar al final / al inicio de la lista
R POP / L POP	Eliminar y retornar el último / primer elemento
L RANGE start stop	Leer subconjunto de la lista (0 -1 = todos)
L LEN	Cantidad de elementos en la lista
L INDEX i	Obtener elemento por índice

Sets	Comandos principales
S ADD / S REM	Agregar / eliminar miembros
S MEMBERS / S CARD	Ver todos los miembros / cantidad
S ISMEMBER	Verificar si un elemento pertenece al set
S INTER / S UNION / S DIFF	Intersección, unión, diferencia entre sets

Sorted Sets	Comandos principales
Z ADD score member	Agregar miembro con puntaje
Z RANGE / Z REVRANGE	Listar por score ascendente / descendente
Z RANGEBYSCORE min max	Filtrar miembros por rango de score
Z SCORE / Z RANK / Z REVRANK	Obtener score o posición de un miembro
Z INCRBY delta member	Incrementar el score de un miembro

Hashes	Comandos principales
H SET key field value	Establecer uno o varios campos
H GET / H GETALL	Obtener un campo / todos los campos
H KEYS / H VALS	Listar solo las keys o solo los values

HEXISTS / HDEL	Verificar existencia / eliminar campo
HINCRBY	Incrementar un campo numérico

Avanzado	Descripción
MULTI / EXEC / DISCARD	Bloque transaccional atómico
SUBSCRIBE / PUBLISH	Sistema de mensajería pub/sub
PSUBSCRIBE patrón	Suscripción con patrón glob
EVAL script numkeys ...	Ejecutar script Lua atómicamente
BGSAVE	Forzar snapshot RDB en background
INFO sección	Estadísticas del servidor (server, stats, persistence)
KEYS patrón / DBSIZE	Listar claves / contar claves totales
TYPE clave	Ver tipo de dato de una clave

Conceptos Clave

Término	Definición
In-memory	Los datos viven en RAM — acceso en microsegundos vs. milisegundos de disco
String	Tipo base de Redis; puede ser texto, número o binario (hasta 512MB)
List	Linked list ordenada; operaciones $O(1)$ en extremos, $O(N)$ en medio
Set	Colección sin duplicados y sin orden; operaciones de conjunto en $O(N)$
Sorted Set (ZSet)	Set donde cada miembro tiene un score float; ordenado automáticamente
Hash	Mapa campo→valor dentro de una clave; eficiente para objetos pequeños
TTL (Time To Live)	Tiempo de vida de una clave; Redis la elimina automáticamente al expirar
Pub/Sub	Mensajería en tiempo real; publisher no sabe quiénes son los subscribers
MULTI / EXEC	Transacción optimista: encola comandos y los ejecuta atómicamente
RDB	Snapshot periódico de la BD a disco (recovery point, no durabilidad completa)
AOF	Append-Only File: registra cada escritura — más durable pero más lento
EVAL / Lua	Scripts atómicos en Lua — ningún otro cliente puede interrumpir

redis-cli	Cliente de línea de comandos incluido en la imagen Docker
-----------	---